Optimisers

SGD vs Adam vs RMSProp on CIFAR-10

A hands-on tutorial comparing three gradient-based optimisers using a fixed MLP on CIFAR-10.

GitHub: github.com/Interloper03/Optimiser_Comparision | Name: Manik Goud Kata | Student ID: 24165296

1. Introduction

Every time you train a neural network, one of the first decisions you make is choosing an optimiser. The optimiser determines how your model updates its weights based on the gradients computed during backpropagation. Make the right choice and your model converges quickly to a good solution. Make the wrong one and you might spend hours waiting for a model that never quite gets there.

This tutorial compares three widely used optimisers — SGD, Adam, and RMSProp — in a controlled experiment. I use the same MLP architecture and the same dataset for all three, so the only variable is the optimiser. The goal is to give you a concrete, visual understanding of how each one behaves, so you can make a more informed choice in your own projects.

The dataset is CIFAR-10, which contains 60,000 colour images (32x32 pixels) across 10 classes — from airplanes to trucks. It is harder than MNIST, which means the optimisers behave more distinctly and the results are more informative.

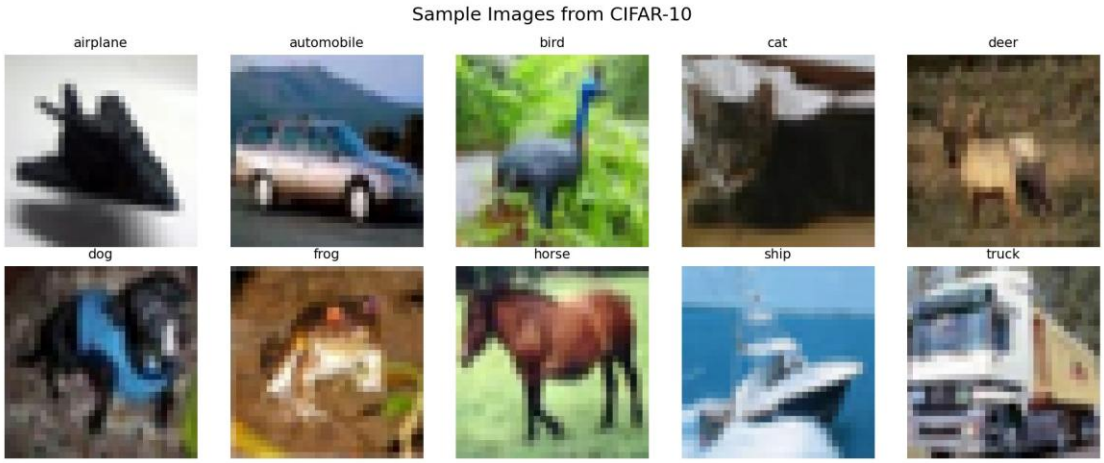

Figure 1. Sample images from each of the 10 CIFAR-10 classes.

2. Experiment Setup

To isolate the effect of the optimiser, every other variable is held constant across all three runs:

Component	Detail
Model	MLP: Flatten → Dense(256, ReLU) → Dense(128, ReLU) → Dense(10, Softmax)
Dataset	CIFAR-10 (50,000 train / 10,000 test, normalised to [0,1])
Loss	Sparse Categorical Cross-Entropy

Epochs	25
Batch size	64
SGD lr	0.01
Adam lr	0.001 (default)
RMSProp lr	0.001 (default)

A slightly higher learning rate of 0.01 is used for SGD (rather than 0.001) because SGD with 0.001 on CIFAR-10 converges far too slowly to be a useful comparison within 25 epochs. Adam and RMSProp use their well-established defaults. A fresh model is built for each run to ensure there is no weight sharing between experiments.

3. How Each Optimiser Works

3.1 SGD — Stochastic Gradient Descent

SGD is the simplest optimiser. At each step it moves every weight in the direction that reduces the loss by a fixed amount controlled by the learning rate (η). Because every parameter gets exactly the same step size regardless of how its gradient has behaved historically, SGD requires careful learning rate tuning. Too low and it crawls; too high and it overshoots and oscillates. This sensitivity is its main drawback.

$$w \leftarrow w - \eta \cdot \nabla L(w)$$

3.2 RMSProp

RMSProp adapts the learning rate per parameter by tracking a moving average of squared gradients. Parameters that have seen large gradients recently get a smaller effective step, and vice versa. This makes it much less sensitive to the choice of learning rate than plain SGD. RMSProp was originally proposed for training recurrent neural networks but works well in many settings.

$$E[g^2]t = \rho \cdot E[g^2]t-1 + (1-\rho) \cdot gt^2$$

$$w \leftarrow w - (\eta / \sqrt{(E[g^2]t + \varepsilon)}) \cdot gt$$

3.3 Adam — Adaptive Moment Estimation

Adam combines RMSProp's adaptive learning rates with momentum. It maintains two running averages: the first moment (mean of gradients, like momentum) and the second moment (mean of squared gradients, like RMSProp). Crucially, it also applies bias correction in early epochs, which prevents the estimates from being too small when training has just started. The default hyperparameters (β1=0.9, β2=0.999) work well across a wide range of tasks, which is why Adam has become the most popular default optimiser in deep learning research [3].

$$mt = \beta1 \cdot mt-1 + (1-\beta1) \cdot gt \qquad [\text{first moment — like momentum}]$$

$$vt = \beta2 \cdot vt-1 + (1-\beta2) \cdot gt^2 \qquad [\text{second moment — like RMSProp}]$$

$$w \leftarrow w - (\eta / (\sqrt{v\hat{}t + \varepsilon})) \cdot m\hat{}t \qquad [\text{bias-corrected update}]$$

Key intuition: SGD treats all parameters equally. RMSProp adapts based on gradient history. Adam does both — it adapts and it has memory of the direction it was moving in.

4. Results

4.1 Training and Validation Curves

Figure 2 shows the training loss (solid lines) and validation loss (dashed lines) over 25 epochs for all three optimisers, alongside the accuracy curves.

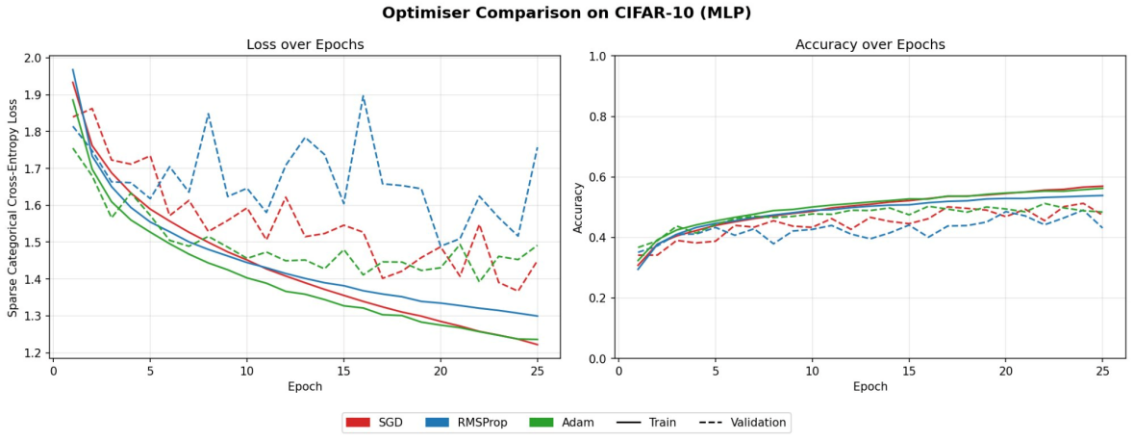

Figure 2. Loss and accuracy over 25 epochs. Solid = training, dashed = validation.

Several things stand out immediately. First, Adam and SGD both show smooth, steadily decreasing training loss, while RMSProp's validation loss is noticeably spiky and unstable — it oscillates throughout training rather than converging cleanly. This is a known behaviour of RMSProp when the effective learning rate fluctuates due to noisy gradient estimates.

Second, all three optimisers converge to similar training accuracy (~54–57%) by epoch 25, but their validation curves tell a different story. Adam produces the most consistent validation performance, while RMSProp's instability is clearly visible in the dashed blue line.

4.2 Final Accuracy

Figure 3 summarises the final train and validation accuracy for each optimiser after 25 epochs.

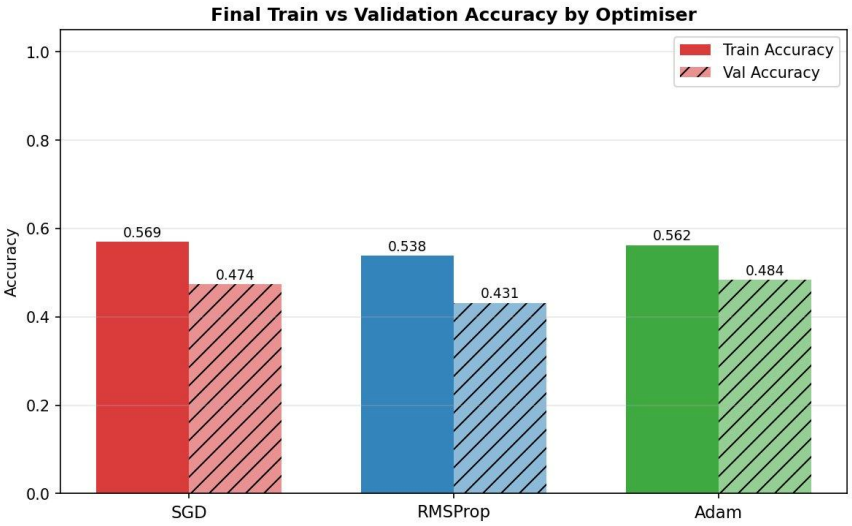

Figure 3. Final training and validation accuracy for each optimiser.

Optimiser	Train Acc	Val Acc	Gap (overfit)
SGD	56.9%	47.4%	9.5%
RMSProp	53.8%	43.1%	10.7%
Adam	56.2%	48.4%	7.8%

Adam achieves the best validation accuracy (48.4%) and the smallest train–validation gap (7.8%), suggesting it generalises slightly better than the other two. SGD has the highest raw training accuracy (56.9%) but does not translate this as well to the test set. RMSProp underperforms both on this task.

4.3 Speed of Convergence

While final accuracy is important, convergence speed matters too — especially when training is expensive. Both Adam and RMSProp reached 40% validation accuracy by epoch 3, while SGD took until epoch 6. This is twice as many epochs just to reach the same threshold. For a task where each epoch is slow, that difference compounds quickly.

Epochs to reach 40% val accuracy: SGD → epoch 6 | RMSProp → epoch 3 | Adam → epoch 3

4.4 SGD's Sensitivity to Learning Rate

One of SGD's defining characteristics is its sensitivity to the learning rate. To demonstrate this, three additional SGD experiments were run with learning rates of 0.001, 0.01, and 0.1.

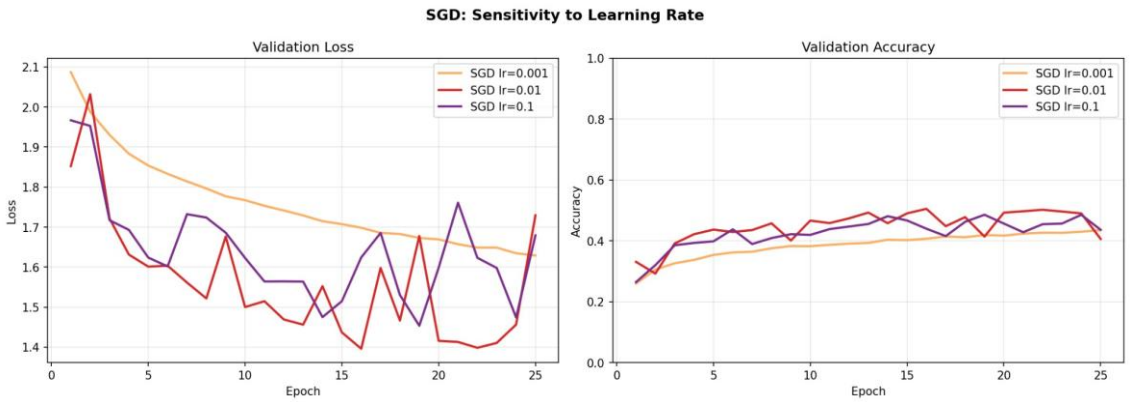

Figure 4. SGD validation loss and accuracy for three different learning rates.

The results reveal something interesting: lr=0.001 converges smoothly but slowly — its loss curve is the cleanest of the three, but it never catches up in accuracy within 25 epochs. In contrast, lr=0.01 and lr=0.1 both converge faster but produce noisier curves, with lr=0.1 showing particularly erratic loss behaviour. All three end up at similar final validation accuracy (~43–44%), but the path there is very different.

This illustrates why SGD is often described as brittle: the right learning rate is dataset- and architecture-dependent, and getting it wrong costs either speed or stability. Adam and RMSProp adapt their step sizes automatically and work well with their default settings.

5. Discussion

The results confirm the established wisdom in the deep learning community, but with some nuance worth highlighting.

Adam is a robust default. It converged fastest, achieved the best validation accuracy, and showed the smallest overfitting gap. This is consistent with its widespread adoption across research and industry. The bias-correction mechanism in Adam gives it a particularly clean start in the first few epochs, which is visible in Figure 2.

RMSProp's instability is worth noting. Despite being theoretically similar to Adam, RMSProp produced noticeably noisier validation curves on this task. Without the momentum term and bias correction that Adam uses, RMSProp's effective learning rate can fluctuate more, leading to the oscillations seen in Figure 2.

SGD should not be written off. While it started slowest and required more learning rate tuning, it ended up with the highest raw training accuracy. There is evidence in the literature that well-tuned SGD with momentum and a learning rate schedule can match or exceed adaptive methods on generalisation for large-scale vision tasks [3]. The key phrase is well-tuned — it simply requires more effort.

It is also worth noting that all three optimisers plateau at relatively modest accuracies (~43–48%) on CIFAR-10. This is expected: a plain MLP with no convolutional layers is not the right architecture for image classification. A CNN would achieve 70–90% on this task. The purpose of this experiment was to isolate and compare optimiser behaviour, not to maximise accuracy.

6. When to Use Which

Situation	Recommendation
Starting a new project	Adam — works well out of the box with default settings
Squeezing out final performance	SGD with momentum + learning rate schedule
Training an RNN or LSTM	RMSProp — it was designed for this setting
Limited compute / time	Adam — fastest convergence in most cases
Noisy / sparse gradients	Adam or RMSProp — both handle this better than SGD

7. Conclusion

This tutorial compared SGD, Adam, and RMSProp in a controlled experiment on CIFAR-10. The key takeaways are: Adam is the most practical default — it converges quickly, generalises well, and requires minimal tuning. RMSProp can be unstable without the momentum and bias correction that Adam provides. SGD is powerful but demanding — it rewards careful tuning but punishes neglect. For most new projects, starting with Adam and then experimenting with SGD if you need to push further is a sound strategy.

The full code for this tutorial, including all experiment steps, is available in the accompanying Jupyter notebook and GitHub repository linked at the top of this document.

8. References

[1] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR 2015. https://arxiv.org/abs/1412.6980

[2] Tieleman, T., & Hinton, G. (2012). Lecture 6.5 — RMSProp. Coursera: Neural Networks for Machine Learning.

[3] Ruder, S. (2016). An overview of gradient descent optimisation algorithms. https://arxiv.org/abs/1609.04747

[4] TensorFlow Documentation — tf.keras.optimizers. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

[5] Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images (CIFAR-10). https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

GitHub Repository: https://github.com/[your-username]/optimiser-showdown